

Security Assessment Report



Aave V4 Config and Deploy Engine

September 2026

Prepared for Aave Labs

Table of Contents

Project Summary **3**
- Project Scope 3
- Project Overview 3
- Protocol Overview 5

Assessment Methodology **8**

Threat and Security Overview **9**
- Findings Summary 14
- Severity Matrix 14

Detailed Findings **15**
- Informational Severity Issues 16
  - I-01: Every role granted to an engine caller must have a zero execution delay 16
  - I-02: The three single field add helpers on the SpokeConfigurator are not reachable 17
  - I-03: The engine has no delegatecall only guard 18
  - I-04: A KEEP_CURRENT placeholder left in a listing is not rejected 19
  - I-05: Misleading sentinel doc 20
  - I-06: Spoke addresses are not deterministic despite the CREATE2 salt scheme 21

Disclaimer **22**

About Certora **22**

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Aave V4 Config Engine	https://github.com/aave/aave-v4	4d86c2d (initial) 459d8ec (fix)	Solidity

Project Overview

This document describes the security review of **Aave V4 Config and Deploy Engine**. The work was undertaken from **September 3rd, 2026** to **September 22nd, 2026**.

The following contract list is included in our scope:

src/config-engine/AaveV4ConfigEngine.sol
src/config-engine/AaveV4Payload.sol
src/config-engine/interfaces/IAaveV4ConfigEngine.sol
src/config-engine/libraries/AccessManagerEngine.sol
src/config-engine/libraries/EngineFlags.sol
src/config-engine/libraries/HubEngine.sol
src/config-engine/libraries/PositionManagerEngine.sol
src/config-engine/libraries/SpokeEngine.sol
src/config-engine/libraries/TokenizationSpokeDeployer.sol
src/deployments/batches/AaveV4AuthorityBatch.sol
src/deployments/batches/AaveV4ConfiguratorBatch.sol
src/deployments/batches/AaveV4GatewayBatch.sol
src/deployments/batches/AaveV4HubInstanceBatch.sol
src/deployments/batches/AaveV4PositionManagerBatch.sol
src/deployments/batches/AaveV4SpokeInstanceBatch.sol
src/deployments/batches/AaveV4TokenizationSpokeBatch.sol
src/deployments/batches/AaveV4TreasurySpokeBatch.sol

src/deployments/libraries/BatchReports.sol
src/deployments/libraries/OrchestrationReports.sol
src/deployments/orchestration/AaveV4DeployBase.sol
src/deployments/orchestration/AaveV4DeployOrchestration.sol
src/deployments/procedures/AaveV4DeployProcedureBase.sol
src/deployments/procedures/deploy/AaveV4AccessManagerEnumerableDeployProcedure.sol
src/deployments/procedures/deploy/hub/AaveV4HubConfiguratorDeployProcedure.sol
src/deployments/procedures/deploy/hub/AaveV4HubDeployProcedure.sol
src/deployments/procedures/deploy/hub/AaveV4InterestRateStrategyDeployProcedure.sol
src/deployments/procedures/deploy/position-manager/AaveV4ConfigPositionManagerDeployProcedure.sol
src/deployments/procedures/deploy/position-manager/AaveV4GiverPositionManagerDeployProcedure.sol
src/deployments/procedures/deploy/position-manager/AaveV4NativeTokenGatewayDeployProcedure.sol
src/deployments/procedures/deploy/position-manager/AaveV4SignatureGatewayDeployProcedure.sol
src/deployments/procedures/deploy/position-manager/AaveV4TakerPositionManagerDeployProcedure.sol
src/deployments/procedures/deploy/spoke/AaveV4AaveOracleDeployProcedure.sol
src/deployments/procedures/deploy/spoke/AaveV4SpokeConfiguratorDeployProcedure.sol
src/deployments/procedures/deploy/spoke/AaveV4SpokeDeployProcedure.sol
src/deployments/procedures/deploy/spoke/AaveV4TokenizationSpokeDeployProcedure.sol
src/deployments/procedures/deploy/spoke/AaveV4TreasurySpokeDeployProcedure.sol
src/deployments/procedures/roles/AaveV4AccessManagerRolesProcedure.sol
src/deployments/procedures/roles/AaveV4HubConfiguratorRolesProcedure.sol
src/deployments/procedures/roles/AaveV4HubRolesProcedure.sol
src/deployments/procedures/roles/AaveV4SpokeConfiguratorRolesProcedure.sol
src/deployments/procedures/roles/AaveV4SpokeRolesProcedure.sol
src/deployments/utils/Logger.sol
src/deployments/utils/MetadataLogger.sol
src/deployments/utils/interfaces/IHubInstance.sol
src/deployments/utils/interfaces/ISpokeInstance.sol
src/deployments/utils/interfaces/ITokenizationSpokeInstance.sol

src/deployments/utils/libraries/BytecodeHelper.sol
src/deployments/utils/libraries/Create2Utils.sol
src/deployments/utils/libraries/DeployConstants.sol
src/deployments/utils/libraries/InputUtils.sol
src/deployments/utils/libraries/Roles.sol

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Aave V4 is a hub-and-spoke lending protocol. Every parameter that governs it is set through restricted functions on the Hub, the Spoke, their configurators and a single AccessManager. The Aave V4 Config Engine is the layer through which governance exercises a defined set of governance configuration actions.

The engine replaces hand-written governance payloads with typed data. A payload declares what should change, as arrays of configuration structs. The engine decides how: which configurator function to call, in which order, and which fields to leave untouched. A proposal reviewer reads a set of structs instead of a sequence of low-level calls.

The engine is stateless. It has no storage, no owner, no roles and no constructor. It is designed to be entered only through delegatecall, so every external call it makes originates from the calling contract and runs with that contract's permissions. In production that contract is the governance Executor.

The system has 3 layers:

- **Payload** (`AaveV4Payload`): an abstract contract the proposal author inherits. It exposes one virtual getter per action type, 21 in total, each returning an empty array by default. The author overrides only the getters the proposal needs. Its `execute()` function runs the action groups in a fixed order and delegatecalls the engine once per non-empty array.
- **Engine** (`AaveV4ConfigEngine`): a dispatcher with one external function per action type. Each forwards to a library. The libraries are linked with external functions, so this second hop is also a delegatecall.

- **Sub-engine libraries** (`HubEngine`, `SpokeEngine`, `AccessManagerEngine`, `PositionManagerEngine`, and `TokenizationSpokeDeployer` inside `HubEngine`): the logic. They resolve identifiers on-chain, merge partial updates against current state, pick the most specific configurator function and make the real call to the target.

Two mechanisms shape every action:

- **Fixed execution order.** `execute()` runs the pre-execution hook, then AccessManager actions, Hub actions, Spoke actions and PositionManager actions, then the post-execution hook. Inside each group the action types also run in a fixed order. Nothing about the order is configurable. Work that must happen out of order is written by hand in the two hooks.

- `KEEP_CURRENT` **sentinels.** Update fields use type-specific `KEEP_CURRENT` values, so that one reserved value per width can mean "leave this field alone". Boolean flags are `uint256` values where 0 is false, 1 is true, the sentinel skips and anything else reverts. Listing structs have no sentinels, because a listing has no current value to keep.

The engine has 4 action groups:

- **AccessManager actions** grant and revoke roles, set a role's admin, guardian, grant delay and label, map function selectors to roles, and set target admin delays. They call the AccessManager directly. They run first, so a role granted in a payload can be used by later actions of the same payload when the role has no grant delay.

- **Hub actions** list assets, update fee, interest rate and reinvestment settings, register spokes on assets, set per-spoke caps and status flags, and apply bulk halts, deactivations and cap resets. They go through the HubConfigurator. An asset listing may also deploy a TokenizationSpoke through CREATE2 and register it on the new asset.

- **Spoke actions** list reserves, update reserve flags and price sources, update the liquidation config, add or edit versioned dynamic reserve configs, and activate or deactivate position managers. They go through the SpokeConfigurator.

- **PositionManager actions** register or deregister a spoke on a position manager and make a position manager renounce its approval for a user. They call the position manager directly and require the Executor to be its owner. Renouncements run before registrations, because renouncing requires the spoke to still be registered.

Users:

• **Governance.** The PayloadsController calls the Executor, which delegatecalls `execute()` on the payload. Because the whole chain is delegatecall, `address(this)` throughout the payload, engine and libraries is the Executor, and `msg.sender` is the PayloadsController. The Executor is the identity every target sees. `msg.sender` is never used. Payload storage is not readable during execution, so action data lives in immutables, constants or literals.

• **Payload authors** inherit `AaveV4Payload`, pass the engine address to its constructor and override the getters that describe the change. The two hooks, `_preExecute()` and `_postExecute()`, hold anything the engine does not express, such as a direct configurator call, a temporary permission grant and its later revocation, or the deployment of a dependency.

• **The Executor** must hold the HubConfigurator domain admin role (200), the SpokeConfigurator domain admin role (400) and the AccessManager admin role (0), which by default is also the admin of every other role. It must own each position manager the payload touches. Every membership must have a zero execution delay and every target it remaps a zero admin delay, since the engine cannot schedule operations.

• **The configurators** are the second permission hop. The HubConfigurator contract holds the Hub configurator role (101) and the SpokeConfigurator contract holds the Spoke configurator role (301). Each action struct names the configurator and the hub or spoke it targets, so one payload may span several hubs and spokes.

• **Protocol users** never call the engine, but their collateral factors, caps, price sources, pause flags and liquidation terms are what it changes.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Scope and Trust Assumptions

The review focused on logic errors and vulnerabilities in the config engine and in how it drives the protocol's restricted functions. This includes the delegatecall execution context, sentinel-based partial updates and their dispatch, action ordering and atomicity, on-chain resolution of identifiers, and the CREATE2 deployment of TokenizationSpoke contracts. The review was conducted through manual analysis of properties, invariants and hypotheses, supported by proof-of-concept tests in the repository's Foundry suite.

As security analysis is inherently creative and any line of code may introduce vulnerabilities, the following sections outline the primary categories of risks evaluated rather than an exhaustive list.

The following are treated as **trust assumptions**, that is they were not verified in depth and must hold for the protocol to operate safely:

- **Governance integrity:** a payload that reaches execution has passed review, and the governance process, Executor and PayloadsController are not compromised. The engine validates the shape of a payload, not its intent. A malicious payload can do through the engine anything the Executor is permitted to do.

- **Production execution topology:** the payload is delegatecalled by the Executor, so `address(this)` is the Executor, and the Executor holds the required roles and ownerships with zero execution delay. The protocol's timelock is the PayloadsController, not the AccessManager, so no delay is expected on the Executor's memberships. A payload called directly, or an engine that itself holds roles, is outside the assumed deployment.

- **Struct-supplied addresses:** the configurator, hub, spoke, authority and position manager addresses in each action struct are correct. The engine keeps no allow-list and calls whatever address it is given, in the Executor's name.

- **External dependencies named in a listing:** an interest rate strategy is bound to the target hub and validates its own parameters, and a price source is a correct feed. The engine only forwards them.

Security Considerations and Threat Analysis

Execution Context. Everything the engine does runs inside the Executor's storage and identity. Threats include:

- A state write in the engine or a library, which would land in the Executor's storage.
- Deriving an owner, a recipient or a permission from `msg.sender`, which is the PayloadsController.
- Reading payload storage during execution, which resolves to the Executor's storage instead, so the payload runs against data it never wrote.
- A revert in a target being swallowed or losing its original reason before it reaches proposal simulation.
- The engine's entry points being callable outside a delegatecall. This is harmless while the engine holds no roles and becomes a privilege escalation path the moment it holds one.

Fidelity of Partial Updates. A field must be written exactly when the author meant it to be written. Threats include:

- A field written when the author meant to skip it. A `uint256` flag left at its default of 0 sets the flag to false, and an address left at zero is a real value.
- A field skipped when the author meant to write it, because the value equals a sentinel.
- A sentinel of a narrower width placed in a wider field, which compiles and is forwarded as a real value.
- Read-modify-write paths merging against stale state or against a target whose ABI differs from the one assumed. The engine does this for interest rate data and dynamic reserve configs, and the configurators do it for every single-field setter.
- Update entries whose fields are all sentinels succeeding with no call and no event.
- Listing structs accepting sentinel values and forwarding them. Some are rejected downstream by a strategy bound or a narrowing cast. An address sentinel is a valid address to the Hub.

Dispatch Correctness. Several actions choose among configurator functions based on which fields are set. Threats include:

- Choosing a function that writes more fields than the struct changed, or fewer.
- Combinations the target rejects, such as replacing an interest rate strategy without complete data, naming the current strategy as if it were new, or relabelling a role whose label state differs from the payload's flag.
- Identifier resolution returning a default value instead of reverting.
- Bulk actions that iterate every spoke of an asset, or every asset of a spoke, affecting more than the author had in mind, including the fee receiver and engine-deployed TokenizationSpokes.

Ordering and Atomicity. A payload is one transaction with a fixed internal order. Threats include:

- An action that depends on a later action in the same payload, such as a reserve listing before the hub asset exists, or a role a later group needs that the AccessManager group revokes first.
- A targeted update and a bulk action on the same subject in one payload, where the fixed order makes the bulk action win.
- Drift between drafting and execution, where a full-struct write re-applies values copied at drafting time and overwrites a change made in between.
- Non-idempotent execution, where a re-run payload reverts on already listed assets, spokes or deployed contracts.

Permission Reach. The engine has no permissions but drives the most privileged ones in the protocol and adds no guard rails. Threats include:

- AccessManager actions that reach further than a reviewer expects: granting or revoking the admin role, setting a role's admin or guardian to the public role, mapping a selector to the public role, or the Executor revoking its own roles.
- An engine action needs two grants. The Executor must hold the configurator role and the configurator must hold the Hub or Spoke role. If either grant is missing, or revoked in the same payload, the whole payload fails.
- A non-zero execution delay on the Executor, or admin delay on a remapped target, turning every engine call into an `AccessManagerNotScheduled` revert that does not name its cause.

- Actions with no engine equivalent pushing governance into hand-written hook code, which is what the engine was built to avoid.

Asset Listing and Contract Deployment. Listing is the only place the engine creates contracts and takes irreversible decisions. Threats include:

- CREATE2 address collisions or front-running of the TokenizationSpoke deployment, leaving a reviewed payload permanently unexecutable.
- The proxy admin owner of a deployed TokenizationSpoke coming from execution context rather than from the payload.
- A partially specified tokenization config being deployed with defaults.
- Side effects not visible in the struct. The fee receiver is registered as a spoke, decimals are read from the token once and never again, and the reinvestment controller cannot be set at listing time.
- Values fixed forever at listing, such as the underlying, its decimals and the asset and reserve identifiers, being wrong with no correction path short of a proxy upgrade.

Interaction with Core Validation. The engine forwards values and leaves validation to the target. Where the target does not validate, the engine is the only line of defence. Threats include:

- The per-spoke configuration a Hub stores without validation, including a risk premium threshold set below a spoke's current ratio, which blocks repayment and liquidation for that pair.
- Editing an existing dynamic reserve config version in place, which changes the terms of every position that uses it immediately. Adding a version affects only positions that later refresh.
- Bulk halt, deactivate and cap-reset actions that apply to a whole asset or spoke in one call but can only be reversed one pair at a time.

Completeness of Coverage. An action the engine cannot express is written outside its review surface. Threats include:

- Restricted functions with no engine action, in particular the emergency levers: halting a spoke across all assets, pausing or freezing all reserves, closing a target, and migrating a contract to a new authority.

• Configurator helpers the engine does not reproduce, such as the ones that copy the latest dynamic config, change one field and add the result as a new version. A payload adding a version must instead restate every field.

• Actions that must enumerate their subjects at drafting time and therefore miss subjects added before execution.

Adversarial Incentives

The engine exposes no permissionless entry point that moves funds, so the adversary model differs from the core protocol's. We consider the following incentive categories:

1. **Griefing governance:** cause a reviewed payload to revert or become permanently unexecutable, for example by occupying a predictable CREATE2 address or by changing state the payload will be merged against.

2. **Silent misconfiguration:** a payload that succeeds but changes nothing, or changes something other than what its reviewer read. The cause may be an honest drafting error, or a proposer who relies on sentinel and ordering semantics to hide intent.

3. **Privilege escalation through a payload:** a proposal presented as a routine parameter change that grants, opens or relocates a permission.

4. **Direct theft:** extract funds belonging to the protocol or its users. Through the engine this is reachable only by an actor who can enter the Executor's context.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	6	6	6
Total	6	6	6

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
I-01	Every role granted to an engine caller must have a zero execution delay	Informational	Fixed
I-02	The three single field add helpers on the SpokeConfigurator are not reachable	Informational	Fixed
I-03	The engine has no delegatecall only guard	Informational	Fixed
I-04	A KEEP_CURRENT placeholder left in a listing is not rejected	Informational	Fixed
I-05	Misleading sentinel doc	Informational	Fixed
I-06	Spoke addresses are not deterministic despite the CREATE2 salt scheme	Informational	Fixed

Informational Severity Issues

I-01: Every role granted to an engine caller must have a zero execution delay

Description:

All engine calls go through `restricted`. If the caller's role in the AccessManager carries an execution delay, OpenZeppelin routes the call to `consumeScheduledOp`, which reverts unless the exact calldata was scheduled beforehand.

The engine has no scheduling action and a payload runs in one transaction, so any engine action under a delayed role always reverts. This is unlikely to happen because the real timelock already lives in the PayloadsController and a second delay inside the AccessManager would just mean waiting twice.

Every grant in the deployment procedures uses a zero delay today. If someone ever sets a delay the failure is an `AccessManagerNotScheduled` error that does not hint at the cause.

Recommendations:

Write the requirement down in the config engine README and assert a zero execution delay in the role procedures when granting to an engine caller, so a bad configuration fails at deployment instead of at proposal execution.

Customer Response:

Fixed in b18b61a (https://github.com/aave/aave-v4/pull/1368) IAaveV4ConfigEngine and AaveV4Payload.execute() state that every role the executor uses through the engine needs a zero execution delay.

Fix Review:

Fix confirmed on PR#1395.

I-02: The three single field add helpers on the SpokeConfigurator are not reachable

Description:

The engine has two dynamic config actions. `executeSpokeDynamicReserveConfigAdditions` calls `addDynamicReserveConfig`, which writes a new key and takes the full config. `executeSpokeDynamicReserveConfigUpdates` calls `updateDynamicReserveConfig`, which writes in place at a key you name, and it does support KEEP_CURRENT because it reads the current values first. So the in place single field helpers `updateCollateralFactor`, `updateMaxLiquidationBonus` and `updateLiquidationFee` are covered in effect, even though the engine never calls them by name. What is not covered is the other three, `addCollateralFactor`, `addMaxLiquidationBonus` and `addLiquidationFee`, which read the latest config, patch one field and add it under a new key. Nothing in the engine does that, so a payload adding a version always restates all three values.

Recommendations:

Consider exposing the three add helpers as engine actions if that shape is meant to be supported. Otherwise note in the README that an addition always states the full config.

Customer Response:

Fixed in b18b61a (https://github.com/aave/aave-v4/pull/1368) the README notes that a dynamic reserve config addition always restates the full config.

Fix Review:

Fix confirmed on PR#1395.

I-03: The engine has no delegatecall only guard

Description:

The engine is meant to be reached only by delegatecall from a payload, so that `address(this)` is the governance executor. Nothing enforces that. Every `execute` function is external with no access control, so the engine can also be called directly at its own address. Today that is harmless because the engine holds no roles and a direct call reverts at the first configurator call. If the engine were ever granted a role, its entry points would become public versions of that role and anyone could call `executeRoleMemberships` to grant themselves permissions with no payload and no vote.

Recommendations:

Add the usual guard to the entry points, an immutable holding `address(this)` at construction plus a require that the current `address(this)` differs from it. Granting the engine a role then becomes harmless by construction.

Customer Response:

Fixed in 5cb526c (https://github.com/aave/aave-v4/pull/1369) every AaveV4ConfigEngine entry point is onlyDelegateCall. The SELF immutable is set in the constructor, so a direct call to the engine address reverts with OnlyDelegateCall().

Fix Review:

Fix confirmed on PR#1395.

I-04: A KEEP_CURRENT placeholder left in a listing is not rejected

Description:

The engine uses placeholder values meaning "leave this alone". They make sense in an update struct but a listing creates something new, so a placeholder there is always a mistake. The usual way it happens is copying an update struct as the template for a listing and missing a field. The engine never checks, it just forwards the value. Placeholders in the interest rate data and in `liquidityFee` happen to revert further down the call, but `feeReceiver` does not.

Recommendations:

Reject placeholders at the top of the listing loop with one require and one custom error. The update path already does exactly this for interest rate data, the listing path is just missing the equivalent. Checking only `feeReceiver` closes the observed hole, but the wider check is cheap and removes the reliance on a bound inside an interest rate strategy the struct lets you replace.

Customer Response:

Fixed in 5cb526c (https://github.com/aave/aave-v4/pull/1369) executeHubAssetListings, executeSpokeReserveListings and executeSpokeDynamicReserveConfigAdditions revert with KeepCurrentNotAllowed() when a field holds its KEEP_CURRENT sentinel. This covers feeReceiver, irStrategy, liquidityFee, irData, tokenization.proxyAdminOwner on asset listings, priceSource and dynamicConfig on reserve listings, and dynamicConfig on additions.

Fix Review:

Fix confirmed on PR#1395.

I-05: Misleading sentinel doc

Files:

src/config-engine/interfaces/IAaveV4ConfigEngine.sol

Description:

There is a misleading doc, in `src/config-engine/interfaces/IAaveV4ConfigEngine.sol` the interface says `type(uint256).max` is the universal `KEEP_CURRENT` value, but role fields are `uint64/uint32` and use `max - 46` / `max - 23`, using `uint64.max` expecting a skip instead forwards `PUBLIC_ROLE`.

Recommendations:

Consider making the doc accurate.

Customer Response:

Fixed in b18b61a (https://github.com/aave/aave-v4/pull/1368) the sentinel docs name the real per-width EngineFlags values and say that type(uint64).max is forwarded as PUBLIC_ROLE.

Fix Review:

Fix confirmed on PR#1395.

I-06: Spoke addresses are not deterministic despite the CREATE2 salt scheme

Description:

The deployment derives a salt per component and deploys through the Safe Singleton Factory, which reads as a guarantee that a given salt produces the same addresses on every chain. That guarantee does not hold for Spokes. `AaveV4SpokeInstanceBatch` deploys the `AaveOracle` with the `new` keyword, so it lands at a plain CREATE address that depends on the batch contract's address, which in turn depends on the deploying account's nonce.

The oracle address is then appended as a constructor argument to the Spoke implementation init code, so the implementation's CREATE2 address moves with it, and the implementation address is embedded in the proxy init code, so the proxy address moves too. The result is that the same inputs and the same salt produce different Spoke addresses depending on how many transactions the deployer had sent beforehand. Every other component in the deployment is genuinely deterministic.

Recommendations:

Consider deploying the `AaveOracle` through `Create2Utils` with a salt derived the same way as the rest of the batch. That restores determinism for the oracle, the Spoke implementation and the Spoke proxy.

Customer Response:

Fixed in 108976a (https://github.com/aave/aave-v4/pull/1370): Spoke addresses no longer depend on the deployer's nonce.

Fix Review:

Fix confirmed on PR#1395.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.